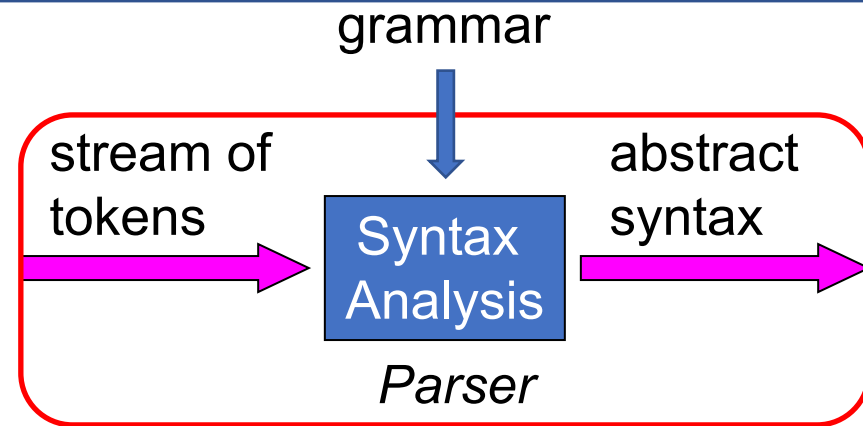


Abstract Syntax

Chapter 4

4.1 Semantic Actions

- A **parser** recognizes whether a sentence belongs to the language of a grammar.



- A **compiler** must do more useful things with that sentence:
 - constructing abstract syntax tree
 - performing semantic analysis
 - generating Intermediate Representation (IR)
- The **semantic actions** of a parser can do useful things with the phrases that are parsed.
- Semantic actions in:
 - Recursive Descent
 - Yacc-Generated Parsers

Recursive Descent

```
enum token {EOF, ID, NUM, PLUS, MINUS, ...};
```

```
 $S \rightarrow E \$$   
 $E \rightarrow T E'$   
 $E' \rightarrow + T E'$   
 $E' \rightarrow - T E'$   
 $E' \rightarrow$   
 $T \rightarrow F T'$   
 $T' \rightarrow * F T'$   
 $T' \rightarrow / F T'$   
 $T' \rightarrow$   
 $F \rightarrow \text{id}$   
 $F \rightarrow \text{num}$   
 $F \rightarrow (E)$ 
```

```
int F_follow[] = { PLUS, TIMES, RPAREN, EOF, -1 };  
void F(void) {switch (tok) {  
    case ID: { advance(); break; }  
    case NUM: { advance(); break; }  
    case LPAREN: { eat(LPAREN); E();  
                  eatOrSkipTo(RPAREN, F_follow);  
                  break; }  
    case EOF:  
    default: printf("expected ID, NUM, or left-  
paren");  
             skipto(F_follow);  
}}}
```

What if we want to evaluate the input string with parsing?

Semantic Actions in Recursive Descent

$S \rightarrow E \$$
 $E \rightarrow T E'$
 $E' \rightarrow + T E'$
 $E' \rightarrow - T E'$
 $E' \rightarrow$
 $T \rightarrow F T'$
 $T' \rightarrow * F T'$
 $T' \rightarrow / F T'$
 $T' \rightarrow$
 $F \rightarrow \text{id}$
 $F \rightarrow \text{num}$
 $F \rightarrow (E)$

```
enum token {EOF, ID, NUM, PLUS, MINUS, ...};
union tokenval {string id; int num; ...};
enum token tok;
union tokenval tokval;
//assume a lookup table mapping identifiers to integers
int lookup(string id) { ... }
int F_follow[] = { PLUS, TIMES, RPAREN, EOF, -1 };
int F(void) {switch (tok) {
    case ID: { int i=lookup(tokval.id); advance();
              return i;}
    case NUM: { int i=tokval.num; advance(); return i; }
    case LPAREN: { eat(LPAREN); int i = E();
                  eatOrSkipTo(RPAREN, F_follow); return i;}
    case EOF:
    default: printf("expected ID, NUM, or left-paren");
             skipto(F_follow); return 0;
}}}
```

Semantic Actions in Recursive Descent

- In a recursive-descent parser, the **semantic actions** are the **values** returned by parsing functions, or the **side effects** of those functions, or **both**.
- Example of side effects:
 - $S \rightarrow id := num$
- For each terminal and nonterminal symbol, we associate a **type of semantic values** representing phrases derived from that symbol.
 - the type is from the implementation language of the compiler

Semantic Actions in Recursive Descent

$S \rightarrow E \$$
 $E \rightarrow T E'$
 $E' \rightarrow + T E'$
 $E' \rightarrow - T E'$
 $E' \rightarrow$
 $T \rightarrow F T'$
 $T' \rightarrow * F T'$
 $T' \rightarrow / F T'$
 $T' \rightarrow$
 $F \rightarrow \text{id}$
 $F \rightarrow \text{num}$
 $F \rightarrow (E)$

```
enum token {EOF, ID, NUM, PLUS, MINUS, ...};
union tokval {string id; int num; ...};

enum token tok;
union tokval tokval;

//assume a lookup table mapping identifiers to integers
int lookup(string id) { ... }

int F_follow[] = { PLUS, TIMES, RPAREN, EOF, -1 };

int F(void) {switch (tok) {
    case ID: { int i=lookup(tokval.id); advance();
               return i;}
    case NUM: { int i=tokval.num; advance(); return i; }
    case LPAREN: { eat(LPAREN); int i = E();
                   eatOrSkipTo(RPAREN, F_follow); return i;}
    case EOF:
    default: printf("expected ID, NUM, or left-paren");
              skipto(F_follow);return 0;
}}}
```

Semantic Actions in Recursive Descent

$$T \rightarrow T * F$$

- The semantic action:

```
int a = T(); eat(TIMES); int b=F(); return a*b;
```

$$E' \rightarrow + T E'$$

$$E' \rightarrow - T E'$$

$$T' \rightarrow * F T'$$

$$T' \rightarrow / F T'$$

There is only right operator

- The semantic action: ?

Semantic Actions in Recursive Descent

$S \rightarrow E \$$
 $E \rightarrow T E'$
 $E' \rightarrow + T E'$
 $E' \rightarrow - T E'$
 $E' \rightarrow$
 $T \rightarrow F T'$
 $T' \rightarrow * F T'$
 $T' \rightarrow / F T'$
 $T' \rightarrow$
 $F \rightarrow \text{id}$
 $F \rightarrow \text{num}$
 $F \rightarrow (E)$

```
int T( void) {switch (tok) {  
    case ID:  
        pass the left operator to T'  
    case NUM:  
    case LPAREN: return Tprime(F());  
    default: print("expected ID, NUM, or left-paren");  
             skipto(T_follow);  
             return 0;  
}}
```

```
int Tprime(int a) {switch (tok) {  
    case TIMES: eat(TIMES); return Tprime(a * F());  
    case PLUS:  
    case RPAREN:  
    case EOF: return a;  
    default: ...  
}}
```

Semantic Actions in Yacc-Generated Parsers

- **{ ... }**: semantic actions
- **\$i**: the semantic values of the *i*_th RHS symbol
- **\$\$**: the semantic value of the LHS nonterminal symbol
- **%union**: difference possible types for semantic values to carry
- **<variant>**: declares the type of each terminal or nonterminal

```
%{ ... %}  
%union {int num; string id;}  
%token <num> INT  
%token <id> ID  
%type <num> exp  
...  
%left UMINUS  
%%  
  
exp: INT {$$ = $1;}  
    | exp PLUS exp {$$ = $1 + $3;}  
    | exp MINUS exp {$$ = $1 - $3;}  
    | exp TIMES exp {$$ = $1 * $3;}  
    | MINUS exp %prec UMINUS {$$ =  
-$2;}
```

Semantic Actions in Yacc-Generated Parsers

- How to implement semantic actions?
- A Yacc-generated parser keeps a stack of semantics values parallel to the state stack.
- When the parser performs a reduction, it must execute the corresponding C-language semantic action.
- How to know $\$i$ for a rule $A \rightarrow Y_1 \dots Y_k$?
 - from the top k elements of the stack
- When the parser pops $Y_k \dots Y_1$ from the symbol stack and pushes A , it also pops k values from the semantic value stack and pushes the value obtained by executing the C semantic action code.

Semantic Actions in Yacc-Generated Parsers

- $$$ = \$1 + \$3$
 - $val = 1 + 6$
- **pop**
 - $\langle exp2, 6 \rangle$
 - $\langle +, NULL \rangle$
 - $\langle exp1, 1 \rangle$
- **push**
 - $\langle exp, val \rangle$

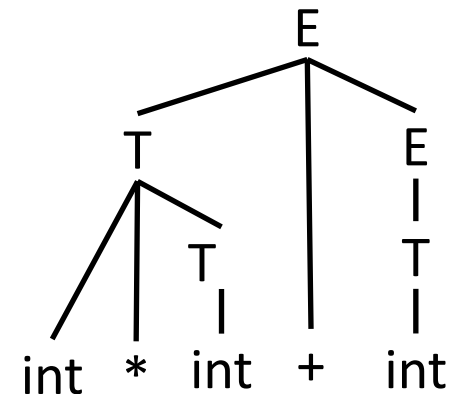
Stack		Input		Action
		1 + 2 * 3 \$		shift
1		+ 2 * 3 \$		reduce
INT				
1		+ 2 * 3 \$		shift
exp				
1		2 * 3 \$		shift
exp	+			
1		* 3 \$		reduce
exp	+			
1		* 3 \$		shift
exp	+			
1		3 \$		shift
exp	+			
1		\$		reduce
exp	+			
1		\$		reduce
exp	+			
1		\$		reduce
exp	+			
1		\$		reduce
exp	+			
7		\$		accept
exp				

Summary: Semantic Actions

- Each terminal and nonterminal may be associated with its own **type of semantic value**.

$A \rightarrow B C D$

- the semantic action must **return a value** whose **type** is the one associated with the **nonterminal A**.
- It can build this value from **the values** associated with the matched terminals and nonterminals **B, C, D**.
- An LR parser does perform reductions, and associated semantic actions, in a **deterministic and predictable order**: a bottom-up, left-to-right traversal of the parse tree.
 - the (virtual) parse tree is traversed in **postorder**



Homework

- 4.2